



Protecting Removable Storage

INSuRE Fall 2023
Midterm Presentation

Sponsor: National Security Agency



Project Mentor
Chris DeWeese - cjdewee@nsa.gov

Faculty Support
Brian Ricks - bwr031000@utdallas.edu

Project Team



University of Texas at Dallas Team 2

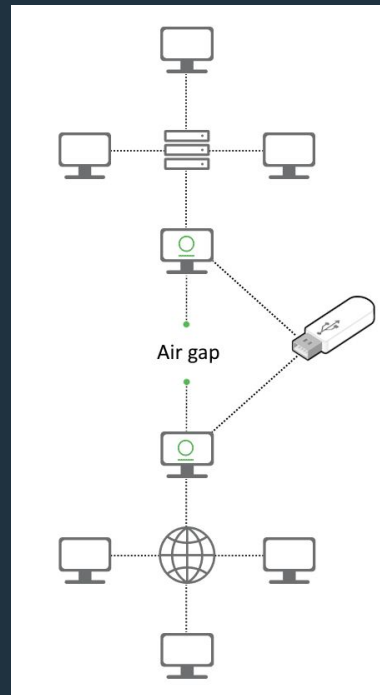
Tri Nguyen	txn180008@utdallas.edu	Computer Science, UTD
Andrei Racila	axr180109@utdallas.edu	Computer Science, UTD
Shreya Soman	svs170430@utdallas.edu	Computer Science, UTD
Shuo-Ting Hsu	sxh200065@utdallas.edu	Computer Science, UTD

Project Motivation



Securing removable storage devices against malware activities and spread.

- Removable storage devices are convenient, capable, and connect to multiple host systems
- Capable of breaching air gap
- Used in virtually all organizations, including governmental and military
- Extensive history as a vector for malware infections



Project Goals



- Understanding the landscape of removable hardware vulnerabilities and defenses
- Develop and evaluate a tamper-resistant software solution to mitigate USB-based attacks



Literature Review



USB Attacks Vectors:

1. Abuse of Human Layer:

- Outsider Threats
- Insider Threats

2. Abuse of Application Layer:

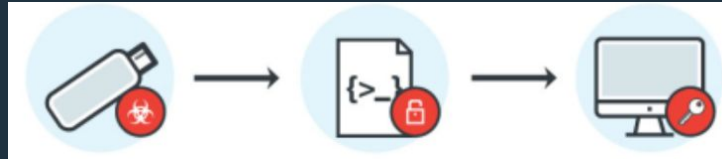
- Code Injection
- Data Exfiltration

3. Abuse of Transport Layer:

- Protocol Masquerading
- Protocol Corruption

4. Abuse of Physical Layer:

- Signal Eavesdropping
- Signal Injection

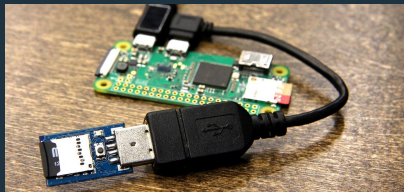


Literature Review



Past USB Attacks

- RubberDucky: emulates the keyboard to inject malicious script
 - USBee, drivebyUSB
 - Psychon: adds keyboard functionality to USB device
- SNOWYDRIVE: create a backdoor, infect other USB devices



Existing Defenses

- White/Blacklisting by device-specific properties
- LBM(Linux eBPF Modules): Filter USB packets in Linux Kernel Space.
- Monitor device behavior in the background
- Fingerprinting



USBGuard & Beamgun

- USB Device authorization via Whitelisting/Blacklisting by vendor, etc.
- Customizable - user can create their own security policy
 - Custom allowable USB list based on properties



Creating an all-rounded system to enhance USB Security!

- Disables suspicious Network Adapters
- Hook keystrokes from USB devices to record keystroke injections
- Steal focus to prevent attacker from making progress

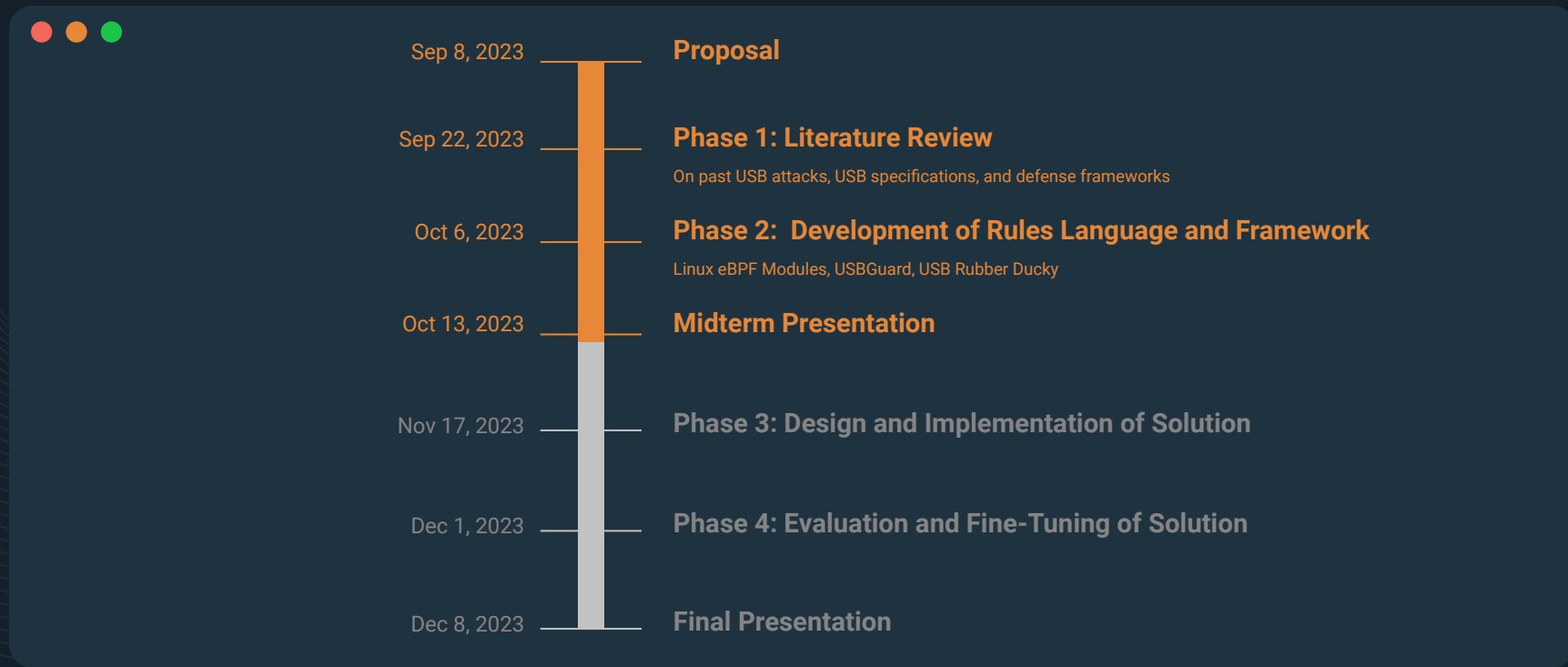


Fingerprinting

- Detection of recognizable patterns of behaviors from BadUSB
 - Rather than only trusting information sent by the USB device
- More reliable and generalizable to different forms of removable storage
- Possible behavioral patterns:
 - File changes
 - Registry key modifications
 - Keyboard input patterns
 - Creation of communication channels to another host
 - Packet structure



Timeline



USB Rubber Ducky

- Physical device: Raspberry pi pico
- Firmware: CircuitPython
- Rubber Ducky Capabilities:
 - Automated Keystroke Injection
 - Payload Executions
 - Scripting
- Payload:
 - Remote access control (open reverse shell for remote user to take over the control of victim's computer)
 - Data exfiltration (store the wifi network passphrase from victim's computer)

```
xushuoting@Tims-MacBook-Pro ~ % nc -l 8888
[ (root@kali-raspberry-pi)-[/home/kali]
# ls
ls
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
USB
Videos
```

LBM (Linux eBPF Modules)

- Tool to allow safe interaction between Linux hosts and peripherals (ex.: USB, Bluetooth, etc.)
- Utilizes eBPF alongside a custom language to provide fast filtering of peripheral packets based on user defined rules
- However, it's focus is not on removable storage and is not easily deployable as it's based on a custom Linux kernel

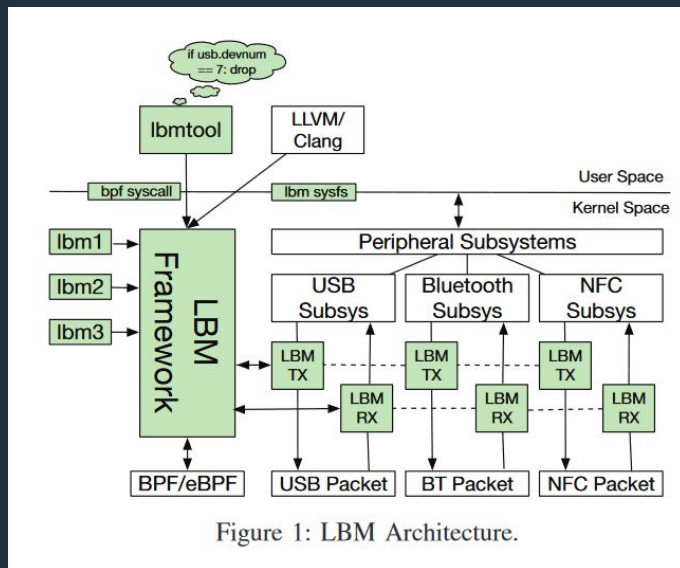


Figure 1: LBM Architecture.

Challenges

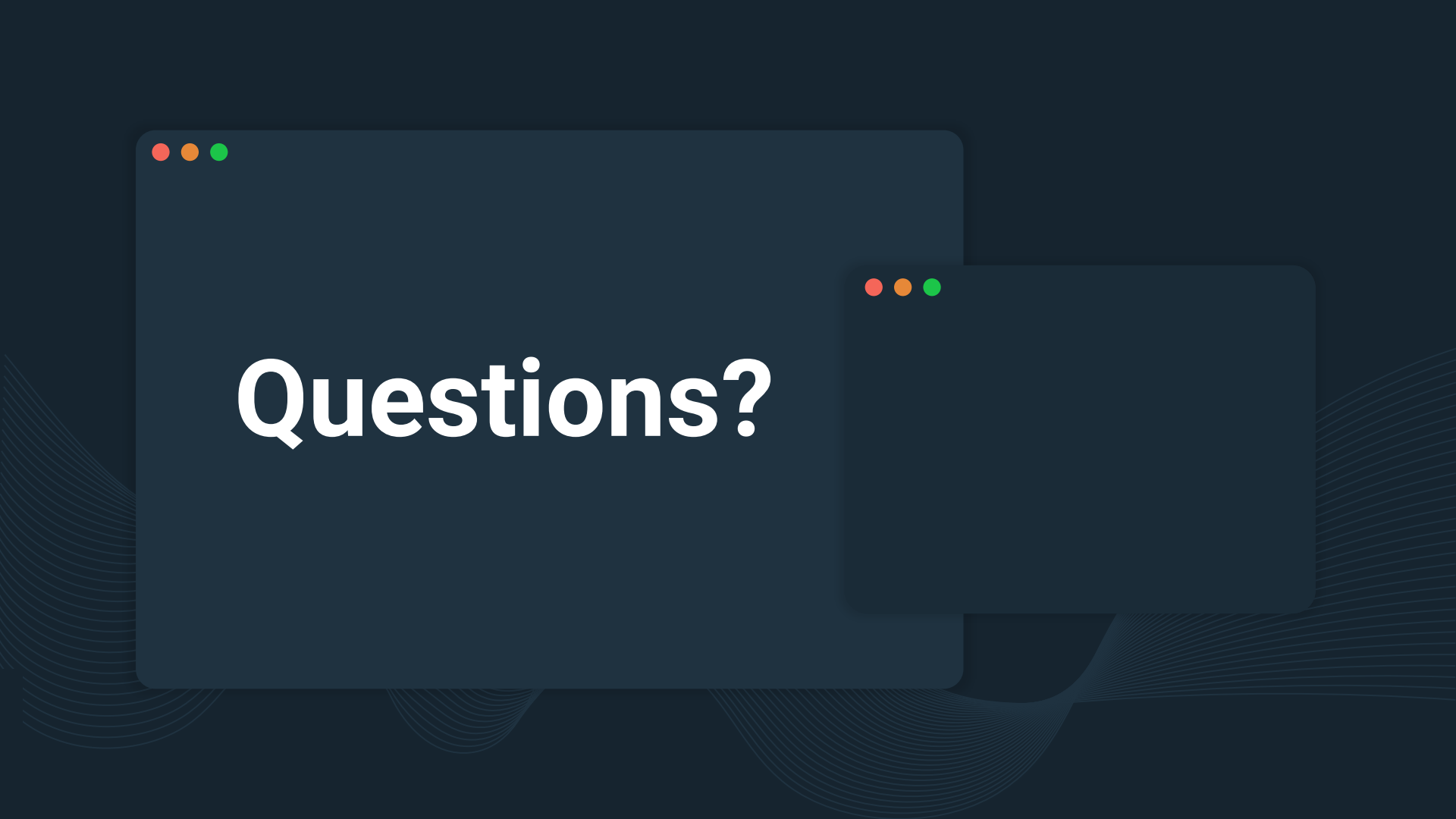


- Setting up LBM:
 - LBM was built directly into a 5-year old Ubuntu kernel
 - Compiling the kernel and getting the bootloader (GRUB) to recognize it took a while
 - No code updates since paper release
- Limited knowledge specific to removable storage among team members
- Removable storage protocols are varied and intricate, making the problem space large and more difficult to address fully.

Next Steps



- Establish an initial defense framework as a basis for implementing a software solution
 - The framework can be continually revised as we develop the solution
- Lift the functionality of LBM core out of the kernel and into something more flexible and extendable, such as kernel modules/probes and eBPF programs.
- Long-term goal: Develop a software solution incorporating what we learned from LBM, USB Specs, Past malware, and other considerations.



Questions?